3D Gaussian Splatting Output Analysis

Scene: truck | Created: 03/11/2026

At a glance

Item	Value
Scene	truck
File	truck_30000.ply
Number of Gaussians N	2,048,162
Properties / Gaussian	62
Bytes / Gaussian	248
SH degree	3
File size	484.4 MB
Check: File size = header + N x bytes/Gaussian	OK
Check: No NaN / infinity	OK
Check: Normals are all 0 (unused)	OK
Check: Opacity after sigmoid lies in (0, 1)	OK
Check: No zero-length quaternions	OK

- 2,048,162 Gaussians, each packing 62 float32 values (248 bytes).

- 484.4 MB in total, and the math checks out to the byte: header + N x bytes per Gaussian.

Anatomy of one Gaussian (.ply layout)

Property	Floats	What it is	How to decode
x y z	3	Center position	as is
nx ny nz	3	Normal (always 0)	skip it
f_dc_0..2	3	Base color (SH degree 0)	RGB = 0.5 + 0.282 * f_dc
f_rest_0..44	45	SH degrees 1-3, laid out R..G..B	used at render time
opacity	1	Opacity (logit)	sigmoid
scale_0..2	3	Axis lengths (log)	exp
rot_0..3	4	Quaternion w, x, y, z	normalize

Contents: A. Where the bytes go | B. Opacity: who is actually visible? | C. Shape check: needles, discs and blobs | D. Spherical Harmonics: is the extra color worth it? | D2. How many SH degrees do we really need? | E. Where the Gaussians live | E2. How crowded is the hotspot? | F. Precision diet: float16 and 8-bit | F2. Position precision vs. Gaussian size | G. Glow-up check: 7k vs 30k iterations | H. Quick wins: what if we just...

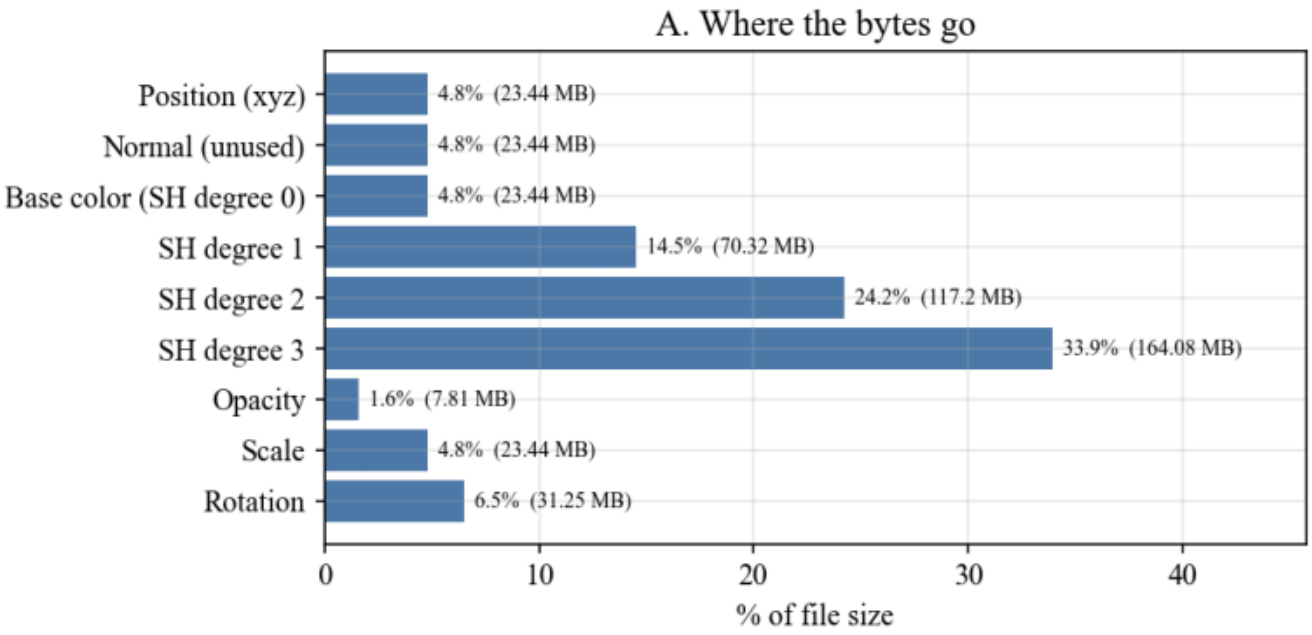
A. Where the bytes go

Every Gaussian carries 62 float32 values. Grouping them by job shows which part is eating the file, which is exactly where compression should start.

TL;DR

- SH degrees 1-3 (view-dependent color) eat 72.6% of the file on their own. Biggest target by far.

- Normals are always 0 yet still take up space. Pure dead weight: dropping them loses nothing.



Group	Floats	Bytes/Gaussian	Total (MB)	% of size
Position (xyz)	3	12	23.44	4.8
Normal (unused)	3	12	23.44	4.8
Base color (SH degree 0)	3	12	23.44	4.8
SH degree 1	9	36	70.32	14.5
SH degree 2	15	60	117.2	24.2
SH degree 3	21	84	164.08	33.9
Opacity	1	4	7.81	1.6
Scale	3	12	23.44	4.8
Rotation	4	16	31.25	6.5
TOTAL	62	248	484.41	100.0

B. Opacity: who is actually visible?

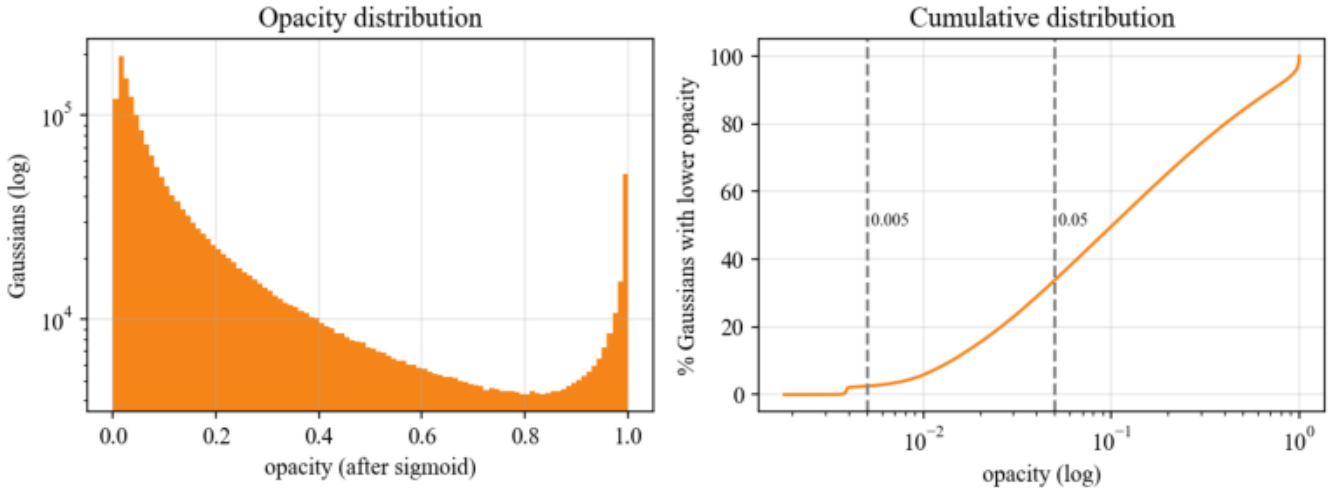
Opacity is stored as a logit and goes through a sigmoid to land in (0, 1). Gaussians that are almost fully transparent barely touch the final image, so they are first in line for pruning (Compact3DGS, Papantonakis et al.).

TL;DR

- Average opacity sits at 0.228, median at 0.101.

- 33.8% of Gaussians are below 0.05, which is basically see-through. Prime pruning candidates.

- 5.9% are close to solid (> 0.9).



Opacity threshold	Gaussians	% of total
< 0.005	51674	2.52
< 0.01	120463	5.88
< 0.05	692382	33.81
< 0.1	1018192	49.71
< 0.5	1719719	83.96
> 0.9 (nearly opaque)	121704	5.94

C. Shape check: needles, discs and blobs

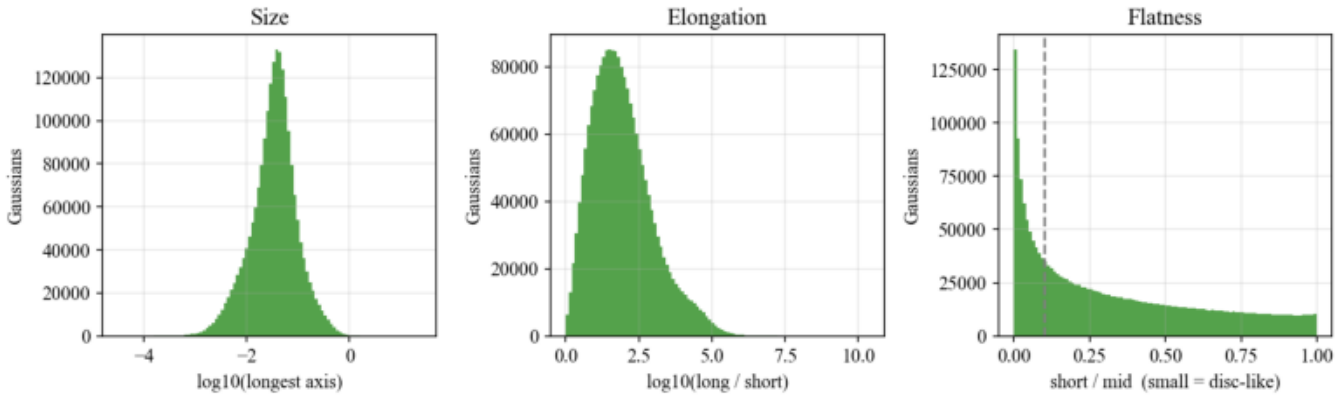
Scale is stored in log space; exp gives the three ellipsoid axes, sorted short < mid < long. A Gaussian counts as a needle when mid < 0.1 x long, a disc when short < 0.1 x mid, and a 3D blob otherwise.

TL;DR

- Typical size: the longest axis has a median of 0.036 and a P99 of 0.385 scene units.

- Shape split: 43.7% needles, 22.2% discs hugging surfaces (same idea as RTG-SLAM), 34.1% 3D blobs.

- The median long/short ratio is 62x. Round Gaussians are rare: just 1.9% have a ratio under 2.



Quantity	P1	P50 (median)	P99	Max
Longest axis	0.00212	0.036	0.385	24.4
Shortest axis	6.09e-07	0.000447	0.0503	0.708
Long/short ratio	1.64	62.1	8.29e+04	2.31e+10
% needle (mid/long < 0.1)		43.7 %		
% disc (short/mid < 0.1)		22.2 %		
% 3D blob (the rest)		34.1 %		
% near-round (long/short < 2)		1.9 %		

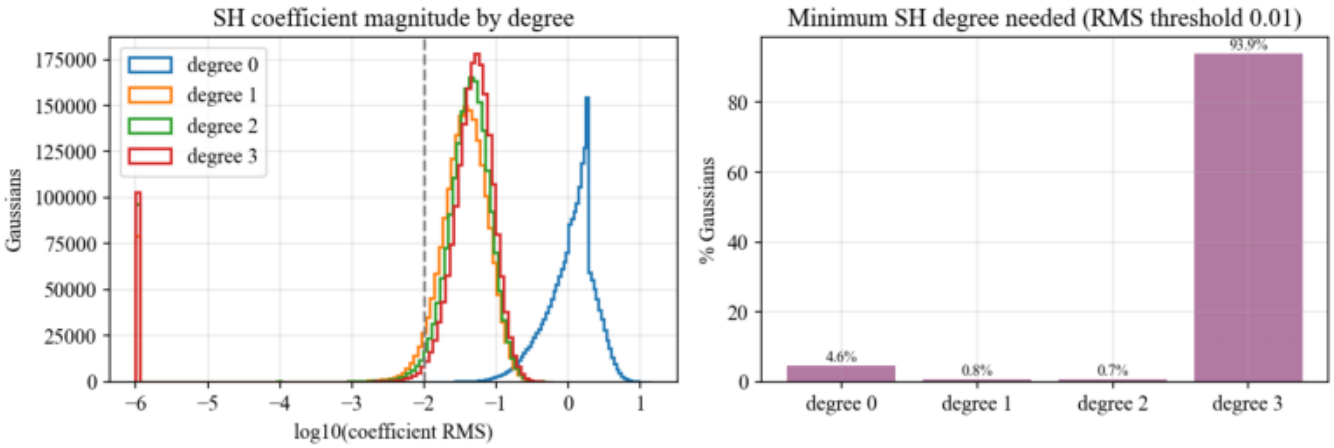
D. Spherical Harmonics: is the extra color worth it?

View-dependent color lives in SH coefficients of degree 0 to 3, 48 numbers in total. When a Gaussian's higher-degree coefficients are basically zero, storing them is pure overhead. This is the idea behind adaptive SH (Papantonakis et al.).

TL;DR

- Base color (degree 0) carries 97.93% of the color energy with just 3 of the 48 coefficients.

- 3.9% of Gaussians have all 45 higher-order coefficients at exactly 0 (the spike at log10 = -6 in the plot). Storing 45 zeros per splat is wasted space.



SH degree	Coefficients (3 channels)	% color energy	Median RMS	% Gaussians RMS < 0.01
0	3	97.93	1.2567	0.0
1	9	0.33	0.0366	7.8
2	15	0.64	0.0421	7.0
3	21	1.1	0.0487	6.1

D2. How many SH degrees do we really need?

A Gaussian 'needs' degree l when the RMS of its degree-l coefficients clears the threshold. This table tests how much the answer moves when the threshold moves.

TL;DR

- The share of Gaussians that only need base color swings from 4.6% (threshold 0.01) to 85.7% (threshold 0.1). The threshold changes the story a lot.

- So the threshold can't be picked by vibes. It has to come from rendering and PSNR, the way Papantonakis et al. tune it on color error across views.

RMS threshold	% need only degree 0	% need only degree 1	% need only degree 2	% need only degree 3
0.01	4.6	0.8	0.7	93.9
0.02	8.0	1.8	2.7	87.5
0.05	43.2	3.5	5.0	48.3
0.1	85.7	1.6	2.1	10.7

E. Where the Gaussians live

Gaussian density projected onto the three coordinate planes. Dense spots mark where the detail is; the faint scatter far out is floaters and background.

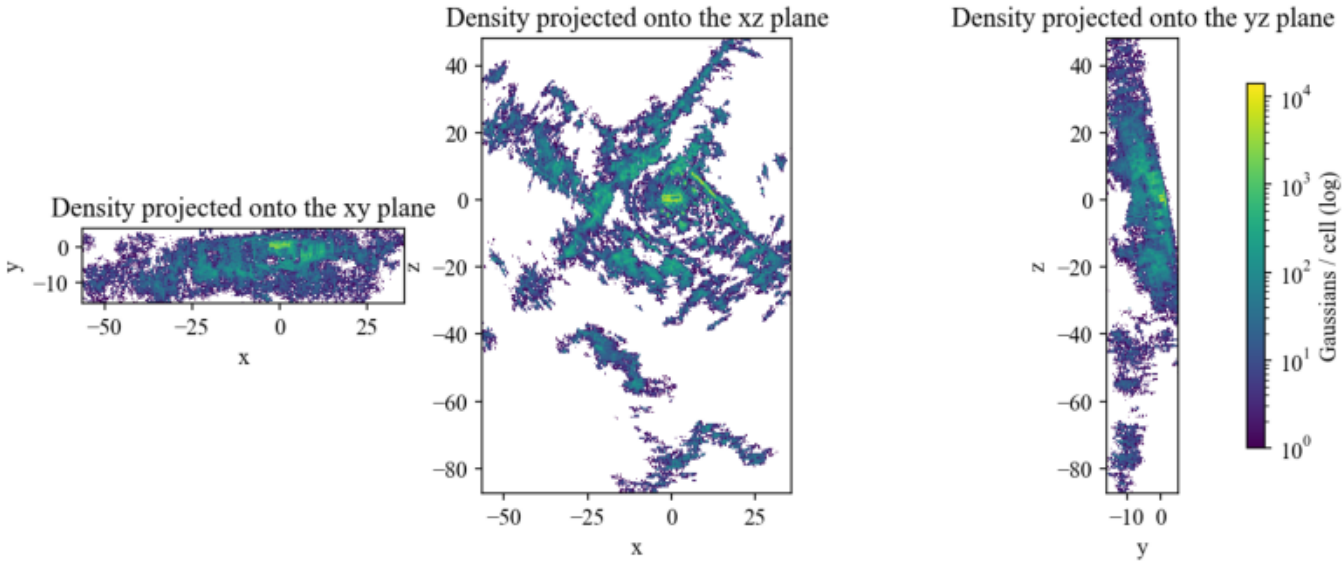
TL;DR

- 98% of Gaussians fit inside a 76.84 x 17.72 x 112.87 box (scene units).

- The spread is very uneven: 30.4% of Gaussians sit in a sphere of radius 6.9 around the hotspot, which is just 0.89% of the volume. That hotspot is the main object, where the cameras spent the most time.

- Far-out stragglers (floaters and distant background): 0.02 % of Gaussians.

- Heads-up: scene units come from COLMAP, they are not meters.



Axis	Min	P1	P99	Max	P1-P99 range
x	-87.71	-48.76	28.08	52.18	76.84
y	-29.59	-14.22	3.50	11.29	17.72
z	-101.79	-76.11	36.76	59.77	112.87
Outside main region					0.02 % of Gaussians

E2. How crowded is the hotspot?

We drop a sphere on the densest region and compare two numbers: the share of Gaussians inside it and the share of scene volume it takes up.

Radius (scene units)	% Gaussians inside	% of P1-P99 box volume
2.8	15.1	0.06
6.9	30.4	0.89
13.8	53.0	7.11

F. Precision diet: float16 and 8-bit

Each attribute group is squeezed into float16 or 8-bit, decoded again and compared with the original. Error is measured against the group's standard deviation: the lower it is, the better the group handles quantization.

TL;DR

- float16 costs about 0.01-0.02% of the standard deviation in every group. Half the size for basically free.

- With 8-bit uniform, the most sensitive group is 'Higher-order SH f_rest' (3.39% of its standard deviation).

- This table is numeric error only. Image quality is a separate question that needs renders and PSNR.

Group	Std dev	float16: mean error	float16: error/std dev	8-bit: mean error	8-bit: error/std dev
Position (scene units)	13.2	0.0014	0.01%	0.157	1.19%
Base color f_dc	1.59	0.00023	0.01%	0.0168	1.06%
Higher-order SH f_rest	0.0628	7.39e-06	0.01%	0.00213	3.39%
Opacity (after sigmoid)	0.279	2.38e-05	0.01%	0.00224	0.80%
Scale (after exp)	0.0542	1.09e-05	0.02%	0.000637	1.17%
Rotation (quaternion)	0.449	5.78e-05	0.01%	0.00196	0.44%

F2. Position precision vs. Gaussian size

For positions, standard deviation is the wrong ruler. What matters is the error compared with the Gaussian's own size: move a splat further than its own width and the geometry falls apart.

TL;DR

- The median Gaussian is only 0.036 across, while the scene spans tens of units, so positions need serious precision.

- 8-bit positions miss by 0.2326 on average, that is 6.47x the size of a typical Gaussian. Splats get shoved out of place and the geometry breaks, which the std-dev view in table F completely hides.

- 16-bit uniform beats float16 on the same bit budget (max error 0.0017 vs 0.0334), because float16 loses precision the further a point is from the origin.

Position encoding	Bits / coordinate	Mean error	Max error	Mean error / median Gaussian size
float16	16	0.0032	0.0334	0.09x
8-bit uniform	8	0.2326	0.4248	6.47x
16-bit uniform	16	0.0009	0.0017	0.03x

G. Glow-up check: 7k vs 30k iterations

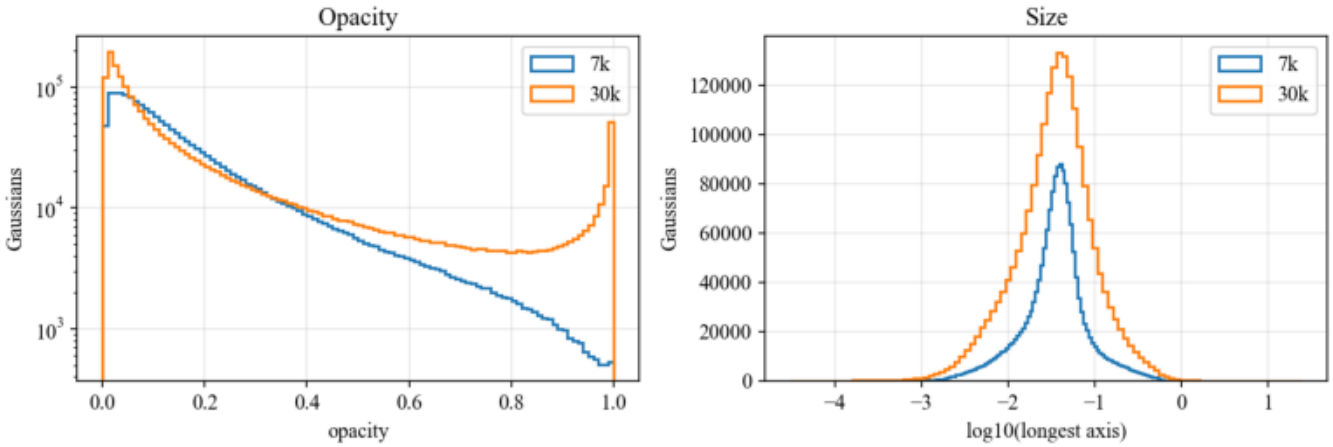
Same scene, two checkpoints. This is what densification does to the Gaussian count, the file size and the shapes along the way.

TL;DR

- The Gaussian count goes from 1,685,982 (7k) to 2,048,162 (30k), x1.21, and the file grows from 399 to 484 MB.

- More training, more extreme shapes: needles 22.5% -> 43.7%, discs 5.4% -> 22.2%.

- Opacity picks a side: near-solid (> 0.9) 0.4% -> 5.9%, near-transparent (< 0.05) 24.1% -> 33.8%.



File	Gaussians	Size (MB)	Mean opacity	% opacity < 0.05	Median longest axis	% needle	% disc
7k iterations	1,685,982	398.8	0.176	24.1	0.0361	22.5	5.4
30k iterations	2,048,162	484.4	0.228	33.8	0.036	43.7	22.2

H. Quick wins: what if we just...

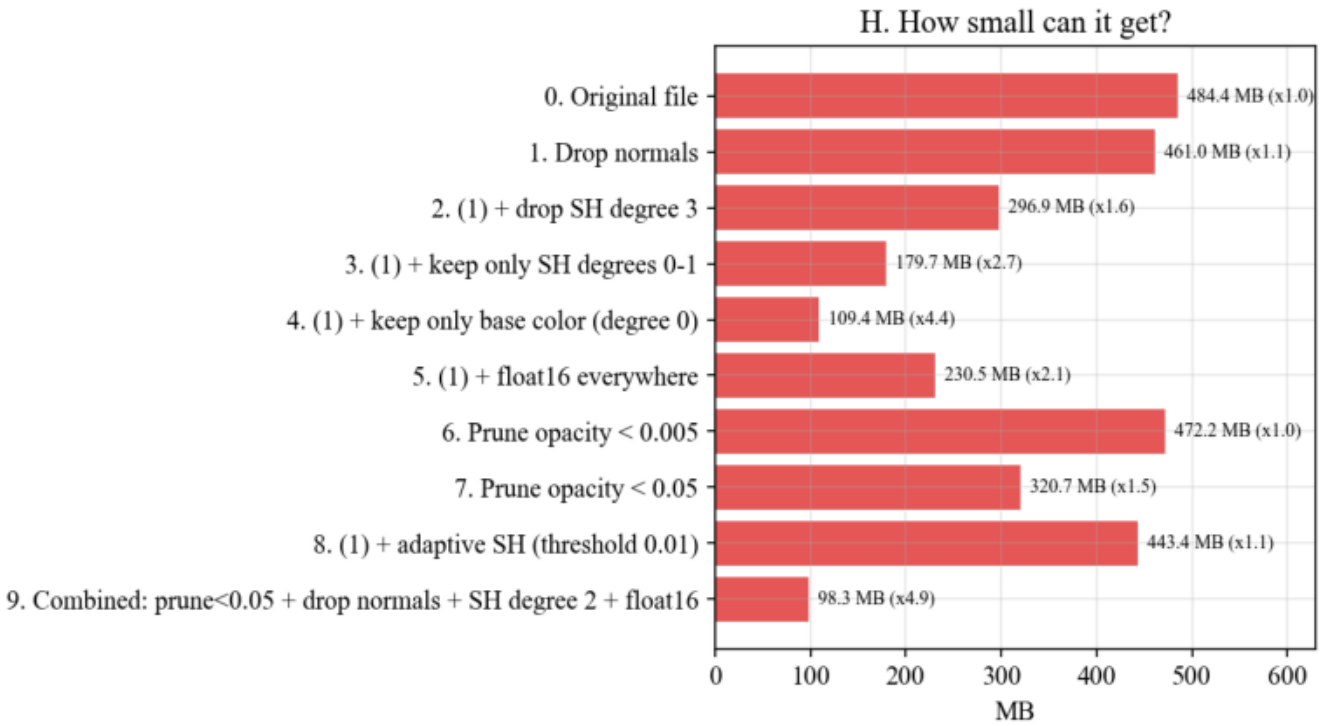
Estimated file size after a few no-brainer moves: drop normals, cut SH degrees, switch to float16, prune by opacity. This covers size only; image quality still has to be checked with renders and PSNR.

TL;DR

- No-brainer moves alone (scenario 9) bring the file down to 98.3 MB, a x4.9 cut.

- For reference, Compact3DGS and Papantonakis et al. reach x15-x30 with codebooks, hash grids and entropy coding.

- What each scenario does to image quality stays unknown until we render and measure PSNR.



Scenario	Size (MB)	Compression (x)	% remaining
0. Original file	484.4	1.0	100.0
1. Drop normals	461.0	1.1	95.2
2. (1) + drop SH degree 3	296.9	1.6	61.3
3. (1) + keep only SH degrees 0-1	179.7	2.7	37.1
4. (1) + keep only base color (degree 0)	109.4	4.4	22.6
5. (1) + float16 everywhere	230.5	2.1	47.6
6. Prune opacity < 0.005	472.2	1.0	97.5
7. Prune opacity < 0.05	320.7	1.5	66.2
8. (1) + adaptive SH (threshold 0.01)	443.4	1.1	91.5
9. Combined: prune<0.05 + drop normals + SH degree 2 + float16	98.3	4.9	20.3